Journal of Machine Learning Research 1 (2026) 1–5 Submitted 7/26; Published

kodama-cpp: Cross-Validated Accuracy Maximization on CPU, CUDA, and Apple Metal

Moussa Kassim[1,2,*] MOUSSA.KASSIM@ICGEB.ORG
Martin Ocharo[1,2,*] MARTIN.OCHARO@ICGEB.ORG
Dalia Ahmed[1] DALIA.AHMED@ICGEB.ORG
Dupe Ojo[1] DUPE.OJO@ICGEB.ORG
Alessia Vignoli[3,4] VIGNOLI@CERM.UNIFI.IT
Leonardo Tenori[3,4,†] TENORI@CERM.UNIFI.IT
Stefano Cacciatore[1,2,†] STEFANO.CACCIATORE@ICGEB.ORG

[1] *Bioinformatics Unit, International Centre for Genetic Engineering and Biotechnology (ICGEB), Cape Town 7925, South Africa*
[2] *Department of Integrative Biomedical Sciences, Institute of Infectious Disease & Molecular Medicine (IDM), University of Cape Town, Cape Town 7925, South Africa*
[3] *Department of Chemistry "Ugo Schiff", University of Florence, Sesto Fiorentino, Italy*
[4] *Magnetic Resonance Center (CERM), University of Florence, Sesto Fiorentino, Italy*
[*] *Moussa Kassim and Martin Ocharo contributed equally.*
[†] *Leonardo Tenori and Stefano Cacciatore are co-corresponding authors.*

Editor: To be assigned

Abstract

KODAMA discovers latent structure by maximizing held-out label predictability. This standalone C++17 implementation is shared by thin R and Python wrappers. Native multicore CPU, NVIDIA CUDA, and Apple Metal backends provide float32 KNN without silent fallback and introduce a label-aware SIMPLS route followed by LDA in latent component space. The library also introduces prediction-guided label evolution and exact-quota landmark projection. In an explicitly exploratory matched CUDA analysis, PLS–LDA KODAMA improved UMAP truth-label silhouette on 6 of 10 datasets (median +0.025, bootstrap 95% interval $[-0.067, +0.084]$); KNN and openTSNE were less favorable. We therefore separate systems validation from external-label diagnostics and make no universal visualization claim.

Keywords: KODAMA, cross-validation, unsupervised learning, heterogeneous computing, manifold learning

1 Introduction

KODAMA treats labels as optimization variables: useful labels can be reproduced on held-out samples. An ensemble of optimized vectors then corrects neighborhood dissimilarities (Cacciatore et al., 2014, 2017). Unlike stability analysis or prediction strength, predictability is inside the search and no observed labels anchor it (Ben-Hur et al., 2002; Tibshirani and Walther, 2005).

Repeated cross-validation makes reusable state a systems concern. Our five contributions are **(1)** a typed C++ core shared by R and Python; **(2)** CPU, CUDA, and Metal under one float32 contract; **(3)** a new label-aware SIMPLS plus latent-space LDA evaluator alongside KNN; **(4)** prediction-guided evolution with adaptive proposals, degeneracy guards, and error-scaled cooling; and **(5)** exact-quota landmark projection for scalable, reproducible subsampling.

2 KODAMA objective and algorithm

For data $X \in \mathbb{R}^{n \times p}$, labels y, folds Π, and classifier family $\mathcal{F}$, let

$$A(y; X, \mathcal{F}, \Pi) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\left[y_i = \widehat{y}_i^{(-\Pi(i))}\right]. \tag{1}$$

KODAMA searches for high A using either KNN or SIMPLS followed by latent-space LDA (de Jong, 1993). Folds stay fixed within each run, and supplied constraint groups move atomically. Each independent run draws exactly L landmarks by population-proportional quotas from a coarse matrix partition; randomized systematic rounding fills the residual quota without replacement. A separate partition initializes the labels, so sampling strata are not optimization targets.

The classifiers share an evolution *scaffold*, not an identical state policy. Both use fixed folds, prediction-guided grouped proposals, the common transition matrix, one new CV pass per proposal, error-scaled cooling, and independent M runs. Let p_k be the class proportions, $D = 1 - \sum_k p_k^2$, $H = -\sum_k p_k \log p_k$, $K_{\text{eff}} = e^H$, and $R = \max\{0, \log[K/\max(1, K_{\text{eff}})]\}$. State selection uses

$$S_{\mathcal{F}}(y) = A(y)\sqrt{D} - \mathbf{1}_{\{\mathcal{F}=\text{PLS-LDA}\}}(1 - A(y)) \max\left\{\frac{H}{\log n}, \frac{R}{1+R}\right\}. \tag{2}$$

Standard `KODAMA.matrix` applies transition-driven coarsening and the subtraction only to PLS–LDA; KNN uses the common diversity term alone. This reflects that LDA fits a mean per active class, whereas KNN has no analogous global class fit. Existing sensitivity results motivate, but do not isolate, this policy; no universal improvement is claimed.

The selected landmark vector is projected to all samples with the same classifier. If a_{ij} is the fraction of projected runs agreeing on graph edge (i, j), KODAMA returns $d'_{ij} = (1 + d_{ij})/a_{ij}^2$ and removes zero-agreement edges. Exact proposal, temperature, quota, and coarsening formulas are given in the supplement.

3 Standalone heterogeneous implementation

The typed C++ API owns folds, proposals, classifiers, landmark projection, graph correction, and metadata; R and Python only convert host objects. CPU provides parallel HNSW over contiguous float32 storage (Malkov and Yashunin, 2020). CUDA and Metal provide exact/IVF search, k-means, label-aware SIMPLS–LDA, PCA, and persistent workspaces. Large accelerator graphs use one resident IVF-Flat index whose list count is independent of the probe bound; a fixed exact pilot raises `nprobe` until recall reaches 0.99. Landmark indices and labels are scattered with epoch tags into a resident all-run label matrix, which

Algorithm 1: KODAMA ensemble
Input: $X, \mathcal{F}, M, T, L, K_0, \Pi, s$. **Output:** $G, C, A_{\text{best}}, Z_U, Z_T$.

$X_{32} \leftarrow \textsc{Float32}(X)$; $(G_0, Z_U, Z_T) \leftarrow \text{KODAMA.}\textsc{Graph}(X_{32})$ once
for $r = 1, \ldots, M$ **do**
  $I_r \leftarrow \textsc{ExactQuotaSample}(\textsc{Partition}(X, K_0), L, s + r)$
  $y \leftarrow \textsc{InitialPartition}(X_{I_r}, K_0)$; $(A, \widehat{y}) \leftarrow \text{CV}(\mathcal{F}, X_{I_r}, y, \Pi_r)$
  for $t = 1, \ldots, T$ **do**
    $y' \leftarrow \textsc{GuidedProposal}(y, \widehat{y}, t, T)$; if $\mathcal{F} =$ PLS-LDA, apply transition coarsening
    $(A', \widehat{y}') \leftarrow \text{CV}(\mathcal{F}, X_{I_r}, y', \Pi_r)$; update by $S_{\mathcal{F}}$ and cooling
  end for
  $C_{\cdot r} \leftarrow \textsc{Project}(\mathcal{F}, X_{I_r}, y_{\text{best}}, X)$
end for
$G \leftarrow \textsc{AgreementCorrectionInPlace}(G_0, C)$; **return** $G, C, A_{\text{best}}, Z_U, Z_T$

Figure 1: Compact pseudocode; PLS–LDA-specific steps are marked.

is consumed directly by agreement-graph correction and downloaded once. Unsupported backends raise instead of silently falling back.

`KODAMA.graph` prepares one graph and backend-matched PCA starts without retaining a caller-visible raw matrix. Its opaque shared owner is represented by an external pointer in R and a capsule in Python, so host index/distance matrices are materialized only on request and a subsequent `KODAMA.matrix` call reuses resident state. `KODAMA.matrix` accepts raw features, this prepared object, or a bare paired $n \times k$ graph. PLS–LDA avoids dense one-hot responses, compacts active labels, and reuses backend-specific fold matrices and float32 workspaces. Metal retains a bounded fold working set and schedules independent M runs against the device memory budget. The API also exposes both CV kernels, both core optimizers, PCA, and float32 UMAP/openTSNE on CPU, CUDA, and Metal (McInnes et al., 2018; van der Maaten and Hinton, 2008). Full architecture, graph-only semantics, numerical safeguards, implementation changes, and wrapper contracts are documented in the supplement.

4 Validation, availability, and scope

Table 1: Systems validation and exploratory matched classic-versus-KODAMA visualization results. Silhouette changes are KODAMA minus classic fastEmbedR medians over seeds 4, 17, and 42; $M = T = 100$.

Scope	Comparison	Backend	Matched data	Result
Kernel	KNNCV, MNIST CPU/device	CUDA	one dataset	16.2x; .974/.973 acc.
Kernel	PLSLDACV, MetRef CPU/device	CUDA	one dataset	4.20x; .992/.993 acc.
Predecessor	MetRef PLS–DA (5 comp.), $M = T = 100$	CPU1	contextual	965.8 s; CV 1.000/.975
Current	MetRef PLS–LDA (50 comp.), $M = T = 100$	CPU4/CUDA	contextual	316.3/270.4 s
Optimizer	shared graph CPU4/device	Metal	15 paired cells	1.83x [1.53,2.12]
Pipeline	flow18 graph, exact/resident-IVF	CUDA	one dataset	296.5/25.9 s; 11.5x
Visualization	KNN KODAMA vs UMAP	CUDA	10 datasets	2/10 improved; −0.094
Visualization	PLS–LDA KODAMA vs UMAP	CUDA	10 datasets	6/10 improved; +0.025
Visualization	KNN KODAMA vs openTSNE	CUDA	10 datasets	0/10 improved; −0.084
Visualization	PLS–LDA KODAMA vs openTSNE	CUDA	10 datasets	3/10 improved; −0.056

Table 1 separates acceleration from downstream structure. The historical MetRef PLS–DA route required 965.8 s at its five-component default; independently retained current 50-component PLS–LDA runs required 316.3 s on CPU4 and 270.4 s on CUDA. This is a

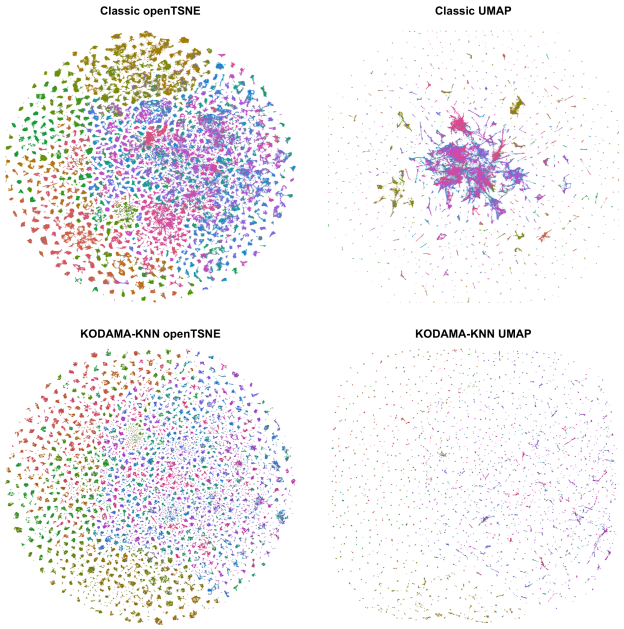

Figure 2: Illustrative ImageNet layouts (n = 1,281,167, 1,000 reference classes; colors repeat), CUDA seed 4. Classic embeddings use the raw feature graph; KODAMA panels use the KNN-corrected graph at $M = T = 100$. Three-seed medians show increased sampled trustworthiness (openTSNE 0.680 to 0.694; UMAP 0.612 to 0.686) but lower truth-label silhouette (0.615 to 0.103; 0.629 to 0.178). Quantitative comparisons use paired metric samples; a frozen rerun will render the same plotting indices in all panels.

contextual product comparison, not an implementation-only speedup: evaluator, components, search, parallelism, and retained seeds differ. On flow18 (n = 1,000,021), resident IVF reduced full graph time from 296.5 to 25.9 s. The matched three-seed archive contains 10 datasets per classifier. PLS–LDA UMAP was the only positive cross-dataset median (+0.025, 95% bootstrap interval $[-0.067, +0.084]$); KNN openTSNE was worse on all 10 (Holm $p = 0.0078$). Figure 2 exposes the corresponding ImageNet trade-off instead of selecting only a favorable example. Labels were withheld during optimization. KODAMA R 2.4.1/2.4 is the sole predecessor; the supplement also reports matched MetRef KNN. All per-dataset timings, sensitivity analyses, and implementation evidence are in the supplement.

Tests cover float32 numerics, backend identity, graph/PCA contracts, visualization initialization, wrappers, and direct public accelerator entry points; physical Metal and current-source CUDA execution are recorded separately from compile-only checks. The MIT repositories are kodama-cpp and kodama-r. Public snapshots checked on 6 August 2026 were `0b8b873` and `7090592`. A tagged confirmatory rerun remains a submission prerequisite; fragmentation, coverage, timings, and limitations are detailed in the supplement.

References

Asa Ben-Hur, Andre Elisseeff, and Isabelle Guyon. A stability based method for discovering structure in clustered data. In *Pacific Symposium on Biocomputing*, pages 6–17, 2002.

Stefano Cacciatore, Claudio Luchinat, and Leonardo Tenori. Knowledge discovery by accuracy maximization. *Proceedings of the National Academy of Sciences*, 111(14):5117–5122, 2014.

Stefano Cacciatore, Leonardo Tenori, Claudio Luchinat, Phillip R. Bennett, and David A. MacIntyre. Kodama: an r package for knowledge discovery and data mining. *Bioinformatics*, 33(4):621–623, 2017. doi: 10.1093/bioinformatics/btw705.

Sijmen de Jong. Simpls: an alternative approach to partial least squares regression. *Chemometrics and Intelligent Laboratory Systems*, 18(3):251–263, 1993.

Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.

Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv:1802.03426*, 2018.

Robert Tibshirani and Guenther Walther. Cluster validation by prediction strength. *Journal of Computational and Graphical Statistics*, 14(3):511–528, 2005.

Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 2008.